

From an Abandoned SSO Login to Executive Profiles Across the Country

By Cat · Translated from the Chinese original

0x01 Preface

In the early phase I combined tools like Lighthouse (灯塔), ffuf, and OneForAll to collect subdomains and pin down the target site. This hunt involved one pitfall after another, but in the end I obtained admin access to a big vendor's backend.

0x02 Pentest Diary — Single Sign-On

Target URL: <https://xxxx.xxxxxxx.com/#/user/login>

(God, one pitfall here: the site only loads in Firefox. Chrome, Edge, the Lenovo bundled browser — none of them worked. It drove me insane...)



Single sign-on — I'm sure the experts here can pop it with their eyes closed, so I won't ramble.

The routine move: brute force.

I set the password list to the top-100 dictionary and the username list to top-10000, then ran a spear-style brute force in Burp Suite.



From testing, a wrong password returns a response whose JSON body echoes back:

```
{
  "sysMessage": "成功",
  "sysCode": 1000,
  "isSuccess": true,
  "businessMessage": "账号密码错误", // wrong account or password
  "businessCode": 1002
}
```

Lucky me. After a long brute-forcing session, several accounts popped out. I then password-sprayed those cracked accounts; the responses came back with Set-Cookie headers and a "success" businessMessage.

Request	Payload	Status	Error	Timeout	Length	Comment
1288		200	☐	☐	1575	
1583		200	☐	☐	1575	
2864		200	☐	☐	1575	
4555	da	200	☐	☐	1575	
2230	ad	403	☐	☐	1551	
2231	adm.	403	☐	☐	1551	
2232	'or 1=1--	403	☐	☐	1551	
2235	admin'or'='	403	☐	☐	1551	
2245	'or'='	403	☐	☐	1551	
2201		400	☐	☐	354	
1	admin	200	☐	☐	332	
10	administrator	200	☐	☐	296	
0		200	☐	☐	296	

Request	Response
1	HTTP/2.0 200 OK
2	Server: nginx
3	Date: Wed, 12 Jan 2022
4	Content-Type: applicati
5	Expires: Wed, 12 Jan 2022 10:10:30 GMT
6	Cache-Control: max-age=0
7	
8	{ "sysMessage": "成功", "sysCode": 1000, "isSuccess": true, "businessMessage": "账号密码错误", "businessCode": 1002 }

You think that's the end?

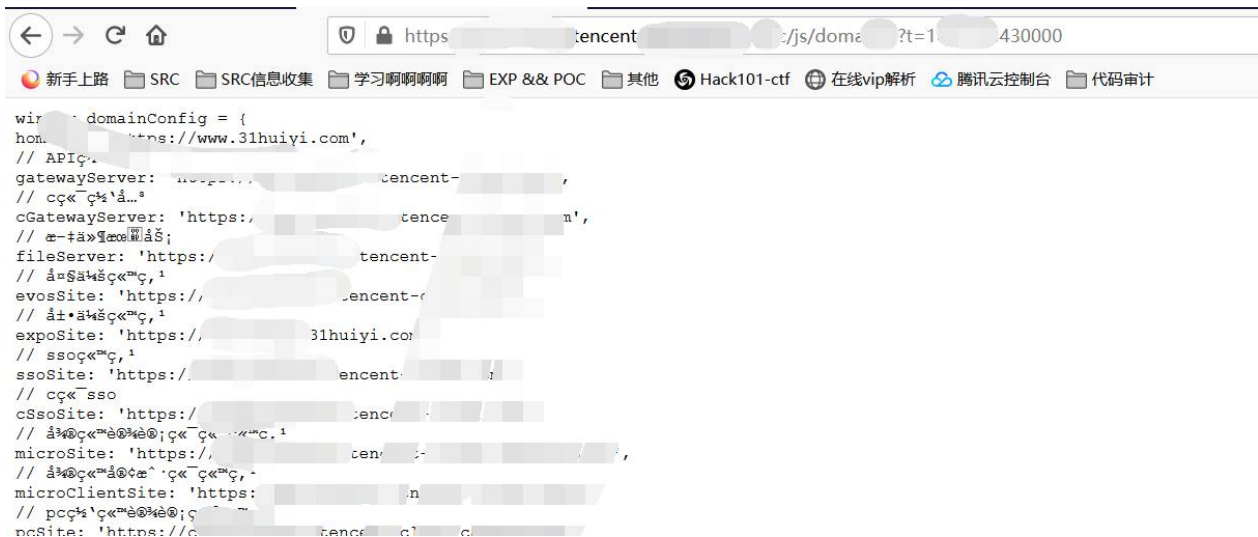
Logging in with a cracked account (amanxxxx / xxxx) did absolutely nothing: the login page stayed the login page. Same account, no reaction — leaving me completely dumbfounded.



Later testing confirmed it: logging in with those cracked credentials never redirects to any backend — it just bounces back to the login page. But after many attempts, these accounts and passwords were definitely real and correct. Not false positives.

0x02 Fuzzing Directories — the Tide Turns

Giving up was not an option. I kept fuzzing directories and discovered a `/js/` path on one host. Brute-forcing the JS directory and extracting what was still reachable eventually yielded this:



Inside a JS file there was a `domainConfig` object full of interesting entries:

```
var domainConfig = {
  homeUrl: 'https://www.31huiyi.com',
  // API
  gatewayServer: 'https://...',
  cGatewayServer: 'https://...',
  fileServer: 'https://...',
  // site
  evossSite: 'https://...',
  exposite: 'https://31huiyi.com/...',
  // SSO
  cssoSite: 'https://...',
  // mobile site
  microsite: 'https://...',
  // design-time site
  ncSite: 'https://...'
};
```

Decoding it in the browser revealed roughly the following: gateways, file servers, expo sites, an SSO site, a mobile site, a design-time site...

With jsfinder plus Burp Suite I enumerated which of those links were still alive.

```
homeUrl: 'https://www
// 网关
gateWayServer: 'https
// c 网关
cGatewayServer: 'http
// 文 服务
fileServer: 'https://
// 大 站点
expose: 'https://cc
// 展 站点
expo: 'https://te
// s 点
ssoSite: 'https://aut
// c sso
cSsoSite: 'https://ca
// 端站点
mobile: 'https://c
// 站站点
microSite: 'http
// p 设计端站点
```

Visiting one of the sites showed this — yet another login page... I'm exhausted. Just kill me now.



But look closely — there's a "Logout" link? I never logged in here. It reminded me of those dead accounts from earlier: the account shown here was the same one. So I ran a test: deleted the browser cookies and revisited.



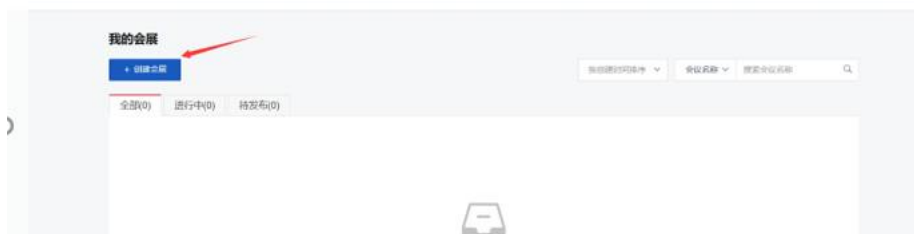
IdentityServer4 again — but this time slightly different from before: the Logout was gone.

I organized my thoughts; the flow was roughly:

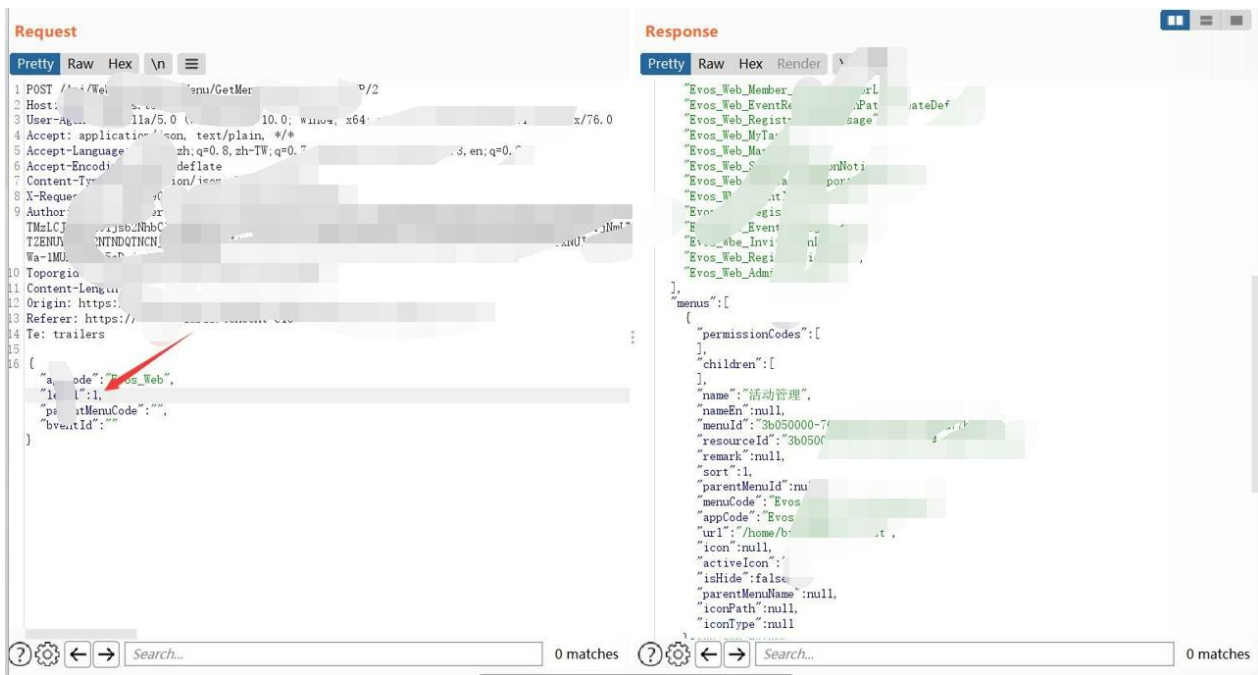
(You log in at <https://xxxx.xxxxxxxx.com/#/user/login> — say with amxxxx / 123456 — it bounces you back to the login page, but here the site displays you as logged in with that very account.)

Repeated testing showed this page only displayed an extra marker indicating the account was logged in; it returned no actual data. (At that point I felt my whole being crack open.)

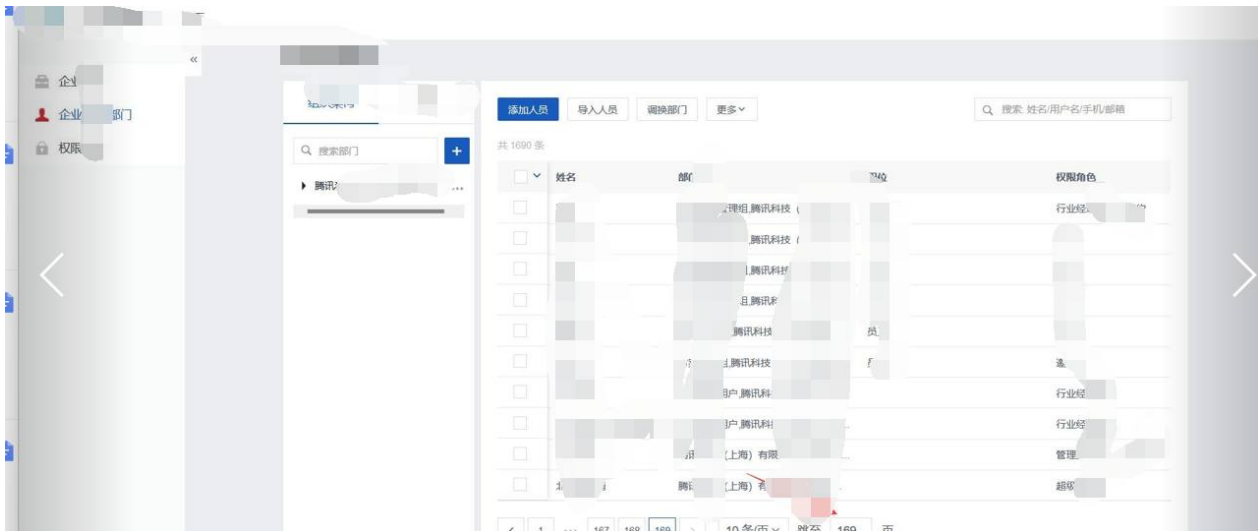
I went back to review those sites, terrified of missing something. Hard work paid off: the interfaces inside those JS files followed a pattern, as shown:



That made things easy. I set the domain prefix to polaxxxx.xxxxxx.com and ran another round of subdomain brute-forcing. Combined with the earlier information, ffuf uncovered an API interface that was no longer in use. The F12 debugger showed the following:



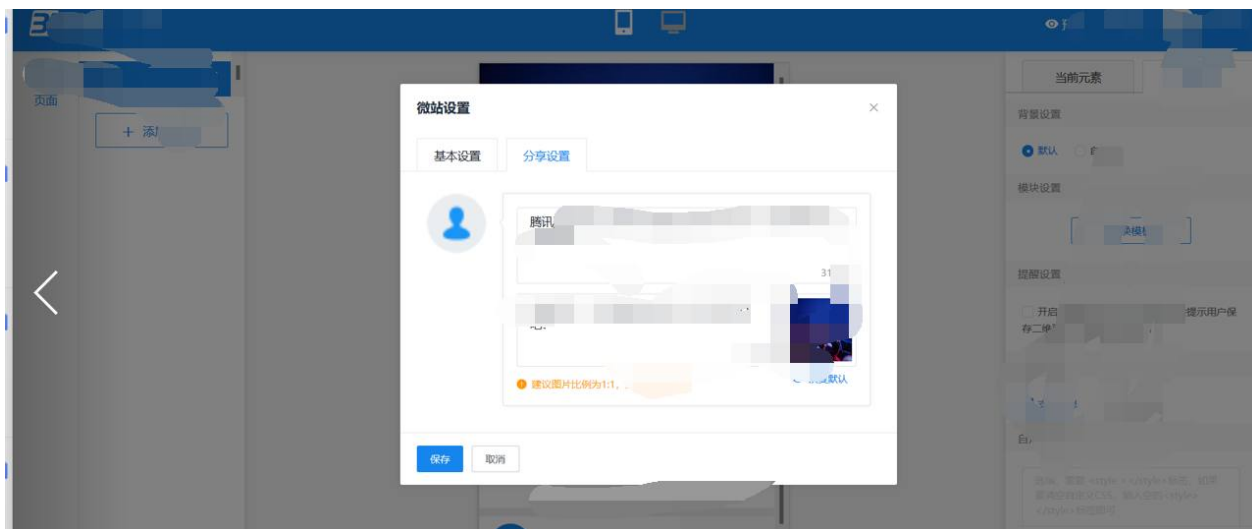
From there it was straightforward: capture the request in Burp Suite, modify the level value inside the JSON — one shot, straight to the soul — and the site opened up, exposing role and permission data (Evos_Web_Role_Member, permissionCodes such as event management and enterprise settings).



The "import personnel" page offered a name/department search — executive profiles from all over the country.



Even a quick lookup via Alipay could confirm the industry managers' information. Finally, using the fact that cookies are shared across this domain's sites, I took over yet another admin backend without logging in at all.



0x03 Summary

From an unusable single sign-on, to ffuf-ing interfaces, to fuzzing usable subdomains, to owning the backend.